

HAETAE 스칼라 Jasmin NTT의 안전성 및 상수 시간 검증

익명 제출

초록

수론 변환(NTT)은 격자 기반 서명 구현에서 짧지만 민감한 계산핵이다. 저수준 구현에서 먼저 확인할 것은 두 가지이다. 실행이 정의된 의미론 안에 머무르는지, 그리고 분기와 주소 계산이 비밀 다항식 계수에 의존하지 않는지이다. 이 논문은 HAETAE의 스칼라 Jasmin 구현 중 NTT, 역 NTT, NTT 영역 기본 곱셈을 대상으로 이러한 구현 수준 성질을 검토한다. 해당 코드는 단계별로 특수화되어 있어 Jasmin 안전성 검사기가 다항식과 트위들 테이블 접근의 경계를 확인할 수 있고, 공개 진입 함수에는 일반 상수 시간 및 추측 실행 상수 시간 주석이 붙어 있다. 검증 논증의 핵심은 호출자 유효성 전체를 갖는 256워드 다항식 포인터, 고정된 공개 반복 일정, 공개 아핀 주소식, 비밀 계수 산술, 그리고 `#init_msf()`가 만드는 진입부 fence이다. 대상 산출물은 `jasminc -checksafety`, `jasmin-ct`, `jasmin-ct -speculative`를 모두 통과한다. 여기에 `hpoly.s`의 바이트 단위 재생성 증거를 결합하면, 생성된 스칼라 x86-64 코드에 대해 소스에서 어셈블리까지 이어지는 안전성 및 분기/주소 누출 흔적 논증을 얻는다. 이것은 임의 어셈블리나 모든 물리적 부채널에 대한 독립 증명이 아니다. 기능적 정확성은 별도의 층위이다. 동반 EasyCrypt 개발은 현재 순방향 및 역방향 NTT 래퍼를 증명하며, 기본 곱셈은 이 논문에서 안전성과 시간 특성만 다룬다.

1 서론

격자 기반 서명은 실행 시간의 상당 부분을 다항식 산술에 사용한다. HAETAE [10]에서 중심이 되는 변환은 $q = 64513$ 위의 256점 NTT이다. 이 변환의 빠른 구현은 단순한 수학 프로그램이 아니다. 실제 코드는 기계어 워드, 테이블 포인터, 반복 카운터, ABI에 노출되는 진입 함수를 다루는 제자리 저수준 루틴이다. 여기서 두 종류의 결합은 특히 치명적이다. 첫째, 테이블이나 다항식에 대한 경계 밖 접근은 이후의 정형 증명이 기대는 실행 모델 자체를 무너뜨린다. 둘째, 비밀 계수에 의존하는 분기나 메모리 주소는 산술 결과가 옳더라도 정보를 누출할 수 있다.

이 논문은 Jasmin [1, 2, 14]으로 작성된 스칼라 HAETAE NTT 구현의 이 두 구현 성질에 초점을 맞춘다. `jasmin/` 아래의 소스는 `poly_ntt_jazz`, `poly_invntt_jazz`, `poly_basemul_jazz` 세 공개 루틴을 구현한다. 해당 코드는 x86-64 어셈블리 `hpoly.s`로 컴파일된다. 같은 소스는 기능적 검증을 위해 EasyCrypt로 추출할 수도 있으나, 이 논문의 주제는 저수준 실행이 정의되어 있고 부채널 규율을 따름을 보이는 컴파일러 수준 보증이다.

이 논문의 주장은 의도적으로 좁고 산출물 중심적이다. 스칼라 Jasmin 구현은 Jasmin 도구 체인이 다음 세 성질을 공개 루틴에 대해 증명할 수 있도록 작성되어 있다.

- **안전성.** 호출자가 유효하고 정렬된 포인터 인자를 제공

주장 영역	이 논문에서 쓰는 증거	주장하지 않는 범위
알고리즘 대상	HAETAE 파라미터, 트위들 테이블 일정, 스칼라 Jasmin 소스	새로운 NTT 알고리즘 또는 벡터화 구현
안전성	<code>jasminc -checksafety</code> 로그와 타입화된 포인터 전체	임의 호출자 메모리의 유효성
일반 및 추측 실행 시간 특성	<code>jasmin-ct</code> , <code>jasmin-ct -speculative</code> , 진입부 fence 형태	모든 미세아키텍처 누출 채널
기능적 정확성	동반 EasyCrypt 순방향/역방향 NTT 정리	기본 곱셈 정확성 또는 전체 서명 보안

표 1: 제출 산출물에 대한 주장 범위.

한다는 전체 아래, 모든 다항식 및 트위들 테이블 접근은 타입이 지정한 256개 원소 영역 안에 있으며, 프로그램이 사용하는 원시 연산은 Jasmin 의미론에서 정의되어 있다.

- **일반 상수 시간.** 분기 조건과 메모리 주소는 공개 반복 카운터, 공개 단계 상수, 공개 포인터 기준 주소에만 의존하며, 다항식 계수에는 의존하지 않는다.
- **추측 실행 상수 시간.** 공개 래퍼는 `#init_msf()`로 추측 실행 경계를 세운다. 따라서 진입 시 `transient` 포인터 입력은 공개 주소 기준값으로 전환되지만, 그 포인터가 가리키는 계수 값은 계속 비밀로 남는다.

표 1는 논문의 주장 범위를 서론에서 먼저 분리해 둔다. 이 논문은 NTT 수학, Jasmin 안전성 및 시간 검사, 컴파일러 보존 논증, 별도의 EasyCrypt 기능적 증명 사이에 놓여 있다. 따라서 각 주장에 어떤 증거를 쓰고, 무엇을 주장하지 않는지를 명시하는 것이 정확성을 높인다.

새로운 NTT 알고리즘, 벡터화 구현, 또는 전체 서명 스킴의 증명을 주장하려는 것이 아니다. 이 논문의 기여는 실제 소스와 생성된 어셈블리 구조에 근거한 스칼라 Jasmin 커널의 구현 수준 보안 논증이다. 특히 보조 함수로 나눈 구현이 Jasmin 안전성 검사, 일반 상수 시간 검사, 추측 실행 상수 시간 검사에 필요한 의무를 어떻게 정확히 드러내는지 기록한다. 이 층위는 기능적 정확성 정리에 앞서 필요하다. 추출된 모델에 대한 정리가 구현 정리가 되려면, 컴파일된 소스가 안전하고 의도한 시간 규율을 보존해야 하기 때문이다.

이 논문의 구체적 기여는 세 가지이다. 첫째, 자동 안전성 및 시간 검사가 관심을 두는 Jasmin 부분언어의 관점에서 스칼라 HAETAE NTT, 역 NTT, 기본 곱셈 커널을 코드 수준으로 설명한다. 둘째, 안전성, 일반 상수 시간, SCT, 컴파일러 보존, 기능적 정확성 주장을 서로 분리하여 각 명제가 어떤 증거에 기대는지 명확히 한다. 셋째, 검사된 소스와 생성된 스칼라 어셈블리에 대한 재현 증거를 산출물에 포함한다. 이후 구성도 이 순서를 따른다. 2장은 구현 대상을 설명하고, 3장부터 7장까지는 검증 모델과 논증을 다룬다. 8장은 재현된 증거를 기록하며, 마지막 절들은 신뢰 경계와 관련 연구를 정리한다.

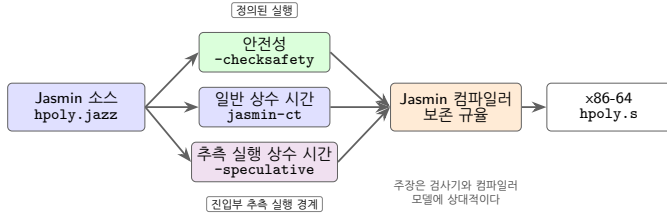


그림 1: 스칼라 HAETAE NTT 산출물의 컴파일러 수준 검증 합성. 안전성, 일반 상수 시간, SCT는 같은 Jasmin 소스 위에서 검사되며, 컴파일러 보존 단계가 그 소스 수준 사실을 생성된 스칼라 어셈블리와 연결한다.

2 구현 대상

산출물은 다섯 개의 소스 및 지원 파일로 이루어진다. `params.jinc`는 변환 크기와 모듈러스를 고정하고, `reduce.jinc`는 Montgomery 축약과 체 곱셈을 구현한다. `zetas.jinc`에는 순방향 및 역방향 트위들 테이블이 들어 있으며, `poly.jinc`는 단계별 커널을 담고 있다. `hpoly.jazz`는 ABI에 공개되는 래퍼와 보안 주석을 포함한다. 표 2은 안전성 및 상수 시간 검증과 직접 관련되는 파일의 역할을 요약한다. 이하의 검증 논증은 `params.jinc`의 문자 그대로의 크기 정의에 의존한다. 모든 다항식 포인터의 타입은 `u32[HAETAE_N]`이고, 각 트위들 테이블은 정확히 256개 원소를 가지며, `HAETAE_Q = 64513`, `HAETAE_N = 256`이다.

파일	검증에서의 역할
<code>params.jinc</code>	공유 상수 <code>HAETAE_Q=64513</code> 및 <code>HAETAE_N=256</code> .
<code>hpoly.jazz</code>	공개 래퍼, <code>ct/sct</code> 주석, 진입부 <code>#init_msf()</code> 호출.
<code>poly.jinc</code>	순방향 NTT, 역 NTT, 역변환 최종 스케일링, 기본 곱셈 루프.
<code>reduce.jinc</code>	고정 폭 워드 위의 Montgomery 산술.
<code>zetas.jinc</code>	고정된 256원소 순방향 및 역방향 트위들 테이블.

표 2: 스칼라 안전성 및 상수 시간 검사에 사용되는 소스 파일.

2.1 알고리즘 및 기능적 맥락

이 구현의 대상은 다음 negacyclic ring이다.

$$R_q = \mathbb{Z}_q[X]/(X^{256} + 1), \quad q = 64513.$$

HAETAE 참조 NTT는 원시 512제곱근 $\zeta_0 = 426$ 을 사용한다 [10, 13]. 추상 체 수준에서 순방향 변환은 ζ_0 의 홀수 거듭제곱들에서 다항식을 평가하며, 출력은 bit-reversal 순서로 놓인다.

$$\tilde{p}[i] = \sum_{j=0}^{255} p[j] \cdot \zeta_0^{(2^{\text{brev}_s(i)+1})j}.$$

역변환은 이에 대응하는 역행렬과 256^{-1} 인자를 적용한다. 동반 기능적 증명에 쓰인 EasyCrypt 개발은 MLKEM/Kyber의 incomplete 128점 구조가 아니라 HAETAE의 full 256 점 변환을 대상으로 한다.

Jasmin 테이블은 이 수학적 대상을 Montgomery word로 구체화한 것이다 [12]. 순방향 테이블의 0번 슬롯은 padding

이고, 1번부터 255번까지는 $\zeta_0^{\text{brev}_s(k)}$ 의 signed Montgomery 대표값을 담는다. 역방향 테이블은 버터플라이 상수를 0번부터 254번까지 명시적으로 저장하고, 255번 슬롯은 최종 스케일링 상수로 둔다. 이 값은 $R = 2^{32}$ 에 대해 $256^{-1}R^2 \bmod q$ 를 나타내는 signed Montgomery word -29720 이다. 따라서 이 논문의 안전성 및 시간 주장은 스칼라 소스가 실제로 읽는 구체적인 테이블 배치에 대한 것이고, 추상 변환과의 연결은 순방향 및 역방향 래퍼에 대한 별도의 EasyCrypt 기능적 정리가 제공한다.

`hpoly.jazz`의 공개 래퍼는 포인터 기반 프로시저 세 개를 제공한다. 포인터 인자에는 일관되게 `p` 접미사를 붙였다. `rp`는 결과 다항식 포인터이고, `ap`와 `bp`는 기본 곱셈의 두 입력 다항식 포인터이다. 기본 곱셈 래퍼는 입력 포인터를 `const`로 표시하고, 결과 포인터만 변경 가능하게 둔다.

Listing 1: `hpoly.jazz`의 공개 래퍼 형태.

```
#[ct = "rp -> rp"]
#[sct = "{ ptr: transient, val: secret } -> { ptr: public, val: secret }"]
export
fn poly_ntt_jazz(reg mut ptr u32[HAETAE_N] rp) -> reg ptr u32[
  HAETAE_N] {
  _ = #init_msf();
  rp = _poly_ntt(rp);
  return rp;
}

#[ct = "rp -> rp"]
#[sct = "{ ptr: transient, val: secret } -> { ptr: public, val: secret }"]
export
fn poly_invntt_jazz(reg mut ptr u32[HAETAE_N] rp) -> reg ptr u32[
  HAETAE_N] {
  _ = #init_msf();
  rp = _poly_invntt(rp);
  return rp;
}

#[ct = "rp * ap * bp -> {ap, bp, rp}"]
#[sct = "
{ ptr: transient, val: secret } *
{ ptr: transient, val: secret } *
{ ptr: transient, val: secret } ->
{ ptr: public, val: secret }
"]
export
fn poly_basemul_jazz(reg mut ptr u32[HAETAE_N] rp,
  reg const ptr u32[HAETAE_N] ap bp) -> reg ptr
  u32[HAETAE_N] {
  _ = #init_msf();
  rp = _poly_basemul(rp, ap, bp);
  return rp;
}
```

래퍼 자체는 의도적으로 얇게 유지되어 있다. 산술은 래퍼가 수행하지 않는다. 래퍼는 보안 경계를 설정한 뒤 내부 커널을 호출한다. 이러한 분리는 검증에 유리하다. 호출자가 제공한 포인터가 검사 대상 코드로 들어오는 지점이 공개 함수이므로, 이곳에 `sct` 계약과 `#init_msf()`를 둔다. 내부 커널은 포인터 기준 주소가 공개 주소 기준값이고 계수 값은 비밀 데이터라는 더 단순한 불변식 아래에서 실행된다.

2.2 Jasmin 구현 규율

이 산출물에서 Jasmin은 고수준 언어라기보다 구조화된 어셈블리 언어로 쓰인다. 검증에 필요한 기계 수준 객체가 소스에 직접 드러난다. 레지스터 변수는 `reg`로 선언되고, 호출자가 넘기는 배열은 `ptr u32[HAETAE_N]` 타입을 갖는다. 변경 가능성도 함수 경계에 나타난다. 예를 들어

```
reg mut ptr u32[HAETAE_N] rp
```

라는 선언은 단순한 설명이 아니다. 안전성 검사기에는 `rp`를 통한 인덱스 접근이 256워드 형태를 갖는다는 정보를 주고, 컴파일러에는 포인터가 레지스터로 전달된다는 사실을 알려 주며, 루틴이 그 포인터를 통해 쓸 수 있다는 점도 기록한다. 그러나 이 선언이 메모리를 할당하거나 그 유효성을 자동으로 보장하는 것은 아니다. 초기 상태는 여전히 포인터 전체에 맞는 유효하고 정렬된 영역을 제공해야 한다. 반면 `poly_basemul_jazz`의 입력 포인터 `ap`와 `bp`는 `reg const ptr u32[HAETAE_N]`로 선언되어 있으며, 실제 구현에서도 읽기 전용 입력으로만 사용된다.

Jasmin 문서는 이 산출물이 쓰는 소스 구조를 프로그램 단위에서 설명한다. Jasmin 프로그램은 `require`로 불러온 파일, 파라미터, 전역 변수, 타입 별칭, 함수로 이루어진다. 표 2의 구성도 바로 이 형식이다. `params.jinc`의 상수는 파라미터이므로 `u32[HAETAE_N]` 같은 배열 크기에 사용할 수 있고, 컴파일러가 초기에 해소한다. `zetas.jinc`의 트위들 테이블은 불변 코드 메모리에 놓이는 전역 배열이다. 배열 문서는 `jzetas`를 읽기 전에 `reg ptr u32[256] zetasp`를 도입하는 이유로 설명한다. `reg ptr`는 전역 배열이나 호출자가 제공한 배열의 주소를 레지스터에 담은 Jasmin 관용구이다.

배열 인덱싱은 0에서 시작하지만, 인덱스는 선언된 원소 크기에 따라 암묵적으로 스케일된다. 따라서 `rp[idx]`는 공개 기준 주소 `rp`에서 $4 \cdot \text{idx}$ 바이트 떨어진 `u32` 셀을 의미한다. 이 점은 안전성에도 시간 논증에도 영향을 준다. 안전성 검사는 단순한 바이트 오프셋이 아니라 $\text{idx} < 256$ 을 필요로 하고, 시간 검사는 생성 주소가 공개 기준 주소와 공개 scaled index로 이루어짐을 본다. 매뉴얼이 지적하듯 레지스터 배열의 run-time 인덱스는 `inlining`, `unrolling`, `constant propagation` 뒤에 정적으로 알려져야 한다. 본 코드는 그런 형태를 피하고, 다항식과 테이블을 공개 run-time 인덱스로 접근할 수 있는 메모리 배열로 둔다. 2026년 Jasmin 자료는 `auto-spill`을 포함한 새로운 레지스터 `spilling` 지원도 설명하지만, 본 산출물은 그 기능에 기대지 않는다. 검증 증거 절 9에 기록한 일반 명령행으로 컴파일한다.

스칼라 타입 문서도 이 구현 해석에 중요하다. 계수는 `machine word`인 `u32`이고, 그 산술은 통상적인 wrapping word 산술이다. 루프와 인덱스 변수는 `word-sized integer`인 `ui64`이다. 이 값들은 소스 의미론에서는 경계가 있는 수학적 정수처럼 동작하고, 이후 컴파일 과정에서 `machine word`로 바뀐다. 이러한 값들을 만드는 연산에는 범위 `assertion`이 따른다. `ui64` 계산이 소스 수준 범위를 벗어난다면 실행은 `stuck` 상태가 되며 안전성 검사기는 이를 거부해야 한다. 이 구현에서는 단계 카운터와 오프셋이 작고 고정되어 있어 해당 범위 의무가 단순하다.

NTT 커널은 저수준 추론을 불필요하게 어렵게 만드는 기능을 피한다. 소스 수준 배열을 새로 만들지 않고, `[p + e]` 같은 원시 바이트 주소 `load`를 사용하지 않으며, 재귀도 쓰지 않는다. 호출하는 보조 함수도 같은 Jasmin 소스 안에서 효과가 보이는 작은 산술 루틴이다. 따라서 관찰 가능한 메모리 연산은 타입이 불은 다항식 접근, 타입이 불은 트위들 테이블 접근, 결과 다항식으로의 저장으로 한정된다. 이 때문에 증명 의무를 소스에서 직접 읽을 수 있다. 모든 주소는 기준 포인터와 공개 워드 인덱스로 만들어지고, 주소가 아닌 데이터 의존성은 레지스터 안의 계수 산술로 남는다.

구현은 공개 NTT 일정과 비밀 데이터 흐름도 분리한다. 길이가 ℓ 인 단계의 제자리 버터플라이는 두 셀 r_i 와 $r_{i+\ell}$ 을

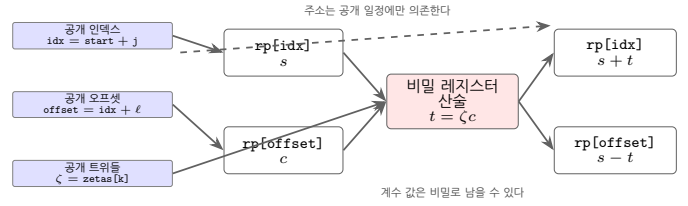


그림 2: 안전성 및 상수 시간 검사가 보는 한 버터플라이 인스턴스. 공개 루프 일정은 접근되는 두 셀과 트위들 테이블 원소를 결정하고, 계수 의존 값은 레지스터 산술과 저장되는 데이터 안에 머문다.

읽고, Montgomery 곱 $t = \zeta r_{i+\ell}$ 을 계산한 뒤, 공개적으로 정해진 같은 두 위치에 $r_i + t$ 와 $r_i - t$ 를 쓴다. Jasmin 코드에서 `block`, `j`, `start`, `idx`, `offset`은 공개 반복 카운터이거나 그 카운터로 만든 식이다. `s`, `coeff`, `t`는 계수 값을 담으므로 비밀일 수 있다. 따라서 검사기는 공개 일정과 비밀 산술 payload가 구분적으로 분리되어 있음을 확인할 수 있다.

단계 특수화는 이러한 분리를 자동 검사기에 맞추기 위한 핵심 구현 장치이다. 간결한 C식 NTT 루프라면 가변 `len`을 두고, `len`, `start`, `j`, 테이블 오프셋의 관계를 중첩 루프 전체에서 추론해야 한다. 본 Jasmin 산출물은 여덟 단계를 여덟 보조 함수로 펼친다. 각 보조 함수에는 리터럴 루프 경계와 리터럴 오프셋이 있으므로, 검사기는 `block < 4`, `j < 32`, `offset = start + j + 32` 같은 단순한 정수 사실을 본다. 이는 NTT 알고리즘을 바꾸는 일이 아니라, 안전성 및 시간 불변식이 각 보조 함수 안에서 지역적으로 보이도록 프로그램을 제시하는 방식이다.

2.3 Montgomery 산술 계층

`reduce.jinc`의 산술 계층은 HAETAE C 참조 구현의 관계를 따른다. 상수 `QINV`는 $q = 64513$ 에 대해 $q^{-1} \bmod 2^{32}$ 이다. 축약 절차는 먼저 입력을 32비트로 절단하고, 감사는 32비트 의미론으로 `QINV`를 곱한 뒤, 그 워드를 부호 확장하여 q 의 배수를 만든다. 이후 해당 값을 빼고 산술 오른쪽 시프트를 32비트 수행한다. Jasmin 코드는 이러한 폭 변환을 모두 명시한다.

Listing 2: `reduce.jinc`의 Montgomery 산술.

```
inline fn __montgomery_reduce(reg u64 a) -> reg u32 {
    reg u32 t32;
    reg u64 t64;

    t32 = (32u)a;
    t32 *= QINV;
    t64 = (64s)t32;
    t64 *= HAETAE_Q;
    a -= t64;
    a >>= 32;
    t32 = (32u)a;
    return t32;
}

inline fn __fqmul(reg u32 a b) -> reg u32 {
    reg u64 ta tb t;
    reg u32 r;

    ta = (64s)a;
    tb = (64s)b;
    t = ta * tb;
    r = __montgomery_reduce(t);
    return r;
}
```

이 계층은 상수 시간 논증에서 중요하다. 계수와 곱은 비밀이지만, 목록 2의 연산은 레지스터 산술이다. 이 연산들은 실행 경로를 선택하지 않고 메모리 주소도 계산하지 않는다. 따라서 Jasmin x86 검사기는 선택된 산술 명령을 상수 시간 명령 모델 안에서 허용하면서도, 같은 비밀 값이 분기나 주소로 흘러가는 경우는 금지해야 한다.

2.4 순방향 변환

순방향 변환은 여덟 개의 고정 보조 함수로 구성되어 있다. 래퍼 `_poly_ntt`는 이들을 다음 단계 길이 순서로 호출한다.

128, 64, 32, 16, 8, 4, 2, 1.

각 보조 함수는 순방향 테이블 `jzetas`를 가리키는 포인터 `zetasp`를 고정하고, 공개 블록 카운터로 반복하며, 공개 트 위들 테이블 원소를 읽은 뒤, 해당 블록 안의 버터플라이에 대해 공개 내부 루프를 수행한다. 첫 번째 보조 함수는 이 구조를 잘 보여 준다.

Listing 3: 첫 번째 단계 특수화 순방향 NTT 보조 함수.

```
fn _poly_ntt_stage_128(reg mut ptr u32[HAETAEN] rp)
-> reg ptr u32[HAETAEN] {
  reg ptr u32[256] zetasp;
  reg ui64 block j start idx offset;
  reg u32 zeta t s coeff;

  zetasp = jzetas;
  block = 0;
  while (block < 1) {
    zeta = zetasp[1 + block];
    start = block;
    start <= 8;
    j = 0;
    while (j < 128) {
      idx = j;
      idx += start;
      s = rp[idx];
      offset = idx;
      offset += 128;
      coeff = rp[offset];
      t = __fqmul(zeta, coeff);
      coeff = s;
      coeff -= t;
      rp[offset] = coeff;
      s += t;
      rp[idx] = s;
      j += 1;
    }
    block += 1;
  }
  return rp;
}
```

자동 검사기에 잘 맞도록 만드는 세부 사항은 두 가지이다. 첫째, 모든 인덱스는 부호 없는 64비트 반복 변수이거나 그 변수들로 만든 식이다. 둘째, 단계 길이가 보조 함수 안에 리터럴로 나타난다. 검사기는 첫 보조 함수에서 `block < 1`, `j < 128`, `offset = idx + 128`을 직접 본다. 가변 `len`과 중첩 루프 사이의 관계를 추론할 필요가 없다. 뒤따르는 보조 함수도 같은 증명 형태를 `block < 2`, `j < 64`; `block < 4`, `j < 32` 등으로 인스턴스화한다. 이는 직접적인 Cooley–Tukey 일정에 대한 알고리즘 변경이 아니라, `jasminc`가 공개 단계 상수를 볼 수 있도록 한 구현 선택이다.

2.5 역변환과 최종 스케일링

역변환도 같은 구현 전략을 반대 단계 순서로 사용한다. 길이 1에서 시작해 길이 128에서 끝나며, 그 뒤 별도의 최종

스케일링 루프를 적용한다. 역변환 보조 함수는 `jzetas_inv`에서 값을 읽으며, 테이블 기준 오프셋은

0, 128, 192, 224, 240, 248, 252, 254

로 고정되어 있다. 최종 스케일링은 `jzetas_inv[255]`를 읽고 모든 계수에 그 고정 인자를 곱한다. 이 마지막 스케일링을 별도 보조 함수로 두면 안전성 논증과 시간 논증이 모두 단순해진다. 이 루프의 유일한 인덱스는 공개 카운터 `j`이고, 가드는 `j < HAETAEN`이다.

역방향 버터플라이에도 데이터 의존 제어는 없다. 코드는 `rp[idx]`와 `rp[offset]`를 읽어 합과 Montgomery 배율이 적용된 차를 계산한 뒤 두 위치에 다시 저장한다. 산술 값은 비밀일 수 있지만, 접근되는 두 위치는 공개 단계 길이, 공개 블록 카운터, 공개 내부 루프 카운터만 으로 결정된다.

2.6 기본 곱셈

기본 곱셈 커널은 메모리 규율을 가장 단순하게 보여 주는 예이다. 이 커널은 공개 256회 루프를 수행하며, 두 입력 다항식에서 각각 한 워드를 읽고, Montgomery 곱을 계산한 뒤, 결과 다항식에 한 워드를 쓴다.

Listing 4: `poly.jinc`의 기본 곱셈.

```
fn _poly_basemul(reg mut ptr u32[HAETAEN] rp,
  reg const ptr u32[HAETAEN] ap bp) -> reg ptr u32
[HAETAEN] {
  reg ui64 i;
  reg u32 a b r;

  i = 0;
  while (i < HAETAEN) {
    a = ap[i];
    b = bp[i];
    r = __fqmul(a, b);
    rp[i] = r;
    i += 1;
  }
  return rp;
}
```

3 Jasmin 검증 모델

Jasmin은 어셈블리 수준의 제어권을 프로그래머에게 주면서도 소스 수준 검증을 가능하게 하도록 설계된 언어이다 [1, 2]. 이 논문에서는 2026년 Jasmin 프레임워크 자료를 현재 기준으로 삼는다. 그 자료는 Jasmin을 예측 가능한 구조화 어셈블리 언어로 설명하고, 소스 수준 안전성 및 상수 시간 검사의 역할을 강조하며, 분기 조건과 메모리 인덱스는 공개여야 한다는 정보 흐름 규칙을 명시한다 [14]. 이전 논문과 발표 자료는 이론적 및 도구적 배경을 제공하고, 2026년 자료는 이 논문이 쓰는 용어와 도구 기대치를 정한다 [11, 8, 6].

두 핵심 원리는 그대로이다. 안전성은 정의성 성질이다. 모든 메모리 접근이 유효해야 하고, 모든 배열 인덱스가 경계 안에 있어야 하며, 원시 연산은 각자의 의미론적 정의역 안에서만 사용되어야 한다. 상수 시간 보안은 흔적 성질이다. 공개 입력이 같은 두 실행은 비밀 상태가 달라도 같은 분기결과 같은 메모리 읽기 및 쓰기 열을 만들어야 한다.

3.1 이론적 근거

이 논문의 이론적 기반은 하나의 단일 정리가 아니라, Jasmin 연구에서 축적된 여러 층의 결과를 이 산출물에 맞게

자료군	이 논문에서의 역할
Jasmin CCS 및 Last Mile 논문	형식 의미론, 소스 수준 검증 성질, 예측 가능한 검증 컴파일, 소스에서 어셈블리로 이어지는 상수 시간 보존 근거.
2021–2026 Jasmin 발표 자료	기존 안전성 검사와 모듈식 안전성 흐름의 구분, 명시적 CT 주석, EasyCrypt 추출, 2026년 기준 용어 정리.
Spectre/SCT 자료	secret/public/transient 세 수준 타입, MSF 규칙, #init_msf() 하향 변환, SCT의 Spectre-v1 범위 [9, 6].
Kyber 구현 정확성 논문	채 수준 기능적 정확성과 구현 안전성 및 시간 주장을 분리한 NTT 검증 선례.
2025 컴파일러 보안 논문	암호학적 보안 보존과 본 산출물 수준 안전성/상수 시간/SCT 검사의 경계를 정하는 관련 연구.

표 3: main.pdf를 제외한 dissertation PDF들이 제공하는 이론적 근거.

조합한 것이다. CCS 2017의 Jasmin 논문은 언어 의미론, 소스 수준 안전성 및 상수 시간 분석, 그리고 컴파일러가 소스 의미와 상수 시간 성질을 보존한다는 기본 구조를 제공한다 [1]. Last Mile 연구는 이 구조를 최적화된 암호 구현의 개발 방법론으로 확장한다. 즉, 저수준 코드는 Jasmin에서 작성하고, 기능적 정확성과 구현 보안 성질은 소스 수준에서 증명하거나 검사한 뒤, 예측 가능한 검증 컴파일러를 통해 그 성질을 어셈블리와 연결한다 [2]. 2021년과 2023년 workbench 자료는 같은 구분을 도구 관점에서 정리한다. 안전성 검사, EasyCrypt 추출을 통한 기능적 추론, 명시적 누출 흔적을 이용한 구현 보안 검사가 서로 다른 층이라는 점이 핵심이다 [11, 8]. 2024–2026년 강의 자료는 이 논문이 따르는 현재 기준이다. 이 자료들은 기능적 정확성 및 안전성, 구현 보안, 암호학적 보안 환원을 구분하고, 최근 안전성 검사 흐름과 분기 조건 및 메모리 인덱스는 공개여야 한다는 소스 수준 정보 흐름 규칙을 명확히 제시한다 [6, 14].

이러한 층위 구분은 이 논문의 주장을 좁고 정확하게 만든다. Kyber 구현 정확성 논문은 Jasmin과 EasyCrypt가 격자 기반 NTT에서 어떻게 사용될 수 있는지 보여 준다. 유한체 표현, 트위들 테이블 일정, 비트 역순, 최적화 루프 순서는 중간 프로그램과 관계적 증명으로 다루고, 구현 보안 검사는 그와 별도의 층으로 남긴다 [3]. 이 논문의 HAETAE 스칼라 NTT에 대한 안전성 및 시간 성질 주장을 기능적 NTT 정확성 증명과 분리하는 이유가 여기에 있다. 한편 최신 컴파일러 보안 보존 논문은 interaction tree와 RHL을 이용해 Jasmin 컴파일러 front-end가 암호학적 보안 실험을 보존한다는 더 강한 종류의 질문을 다룬다 [4]. 그 결과는 이 논문의 위치를 설명하는 관련 연구이지, HAETAE 서명 보안, 컴파일러 back-end 보안, 또는 이 산출물의 부채널 보안을 자동으로 제공하는 전제가 아니다.

3.2 이 논문에서 사용하는 위협 및 누출 모델

여기서 고려하는 구현 수준 공격자는 Jasmin x86 검사기가 모델링하는 누출 흔적을 관찰한다고 본다. 임의의 물리적 부채널을 모두 다룬다는 뜻은 아니다. 공개 입력은 선택된 루팅, 포인터 기준 주소, 반복 경계, 테이블 배치, 단계 상수, 고정된 트위들 테이블 위치이다. 비밀 상태는 다항식 계수와 그 계수에서 유도된 모든 산출 값이다. 일반 상수 시간 검사는 분기 결정, 루프 가드, 메모리 주소, 배열 인덱스, 그리고 해당 아키텍처 모델에서 상수 시간으로 분류되지 않은 명령 피연산자를 관찰 가능한 것으로 본다. 따라서 이 논문의

불변식은 명확하다. 계수를 바꾸면 레지스터 값과 저장되는 계수 값은 달라질 수 있지만, 실행되는 제어 흐름 경로와 프로그램이 접근하는 메모리 위치의 열은 달라지지 않아야 한다.

추측 실행 검사기는 이 모델을 Jasmin 도구가 사용하는 추측 실행 상수 시간/Spectre-v1식 의미에서 확장한다. 공개 래퍼는 transient 포인터 입력을 받고, 내부 커널을 호출하기 전에 경계에서 fence를 둔다. 내부 커널에서는 포인터 기준 주소와 인덱스가 공개이고, 로드된 계수 값은 계속 비밀이다. 이는 모델에 상대적인 주장이다. 전력, 전자기파, 주파수 변화, 캐시 교체 정책 상태, 검사기가 표현하지 않는 프리패치 동작 같은 모든 미세아키텍처 채널을 포괄한다는 뜻이 아니다. 또한 호출자가 유효하고 정렬된 다항식 영역을 넘겨야 한다는 일반 ABI 전제도 제거하지 않는다.

3.3 안전성 검사의 원리

안전성 검사기는 Jasmin 소스 의미론에 대한 추상 해석기이다. 이 검사는 NTT가 수학적으로 올바른 다항식 변환을 계산한다는 사실을 증명하지 않는다. 대신 그러한 기능적 증명이 전제하는 실행 의미론 밖으로 프로그램이 벗어나지 않음을 보인다. 본 산출물에서 중요한 안전성 전제는 공개 타입에서 온다. 호출자는 다항식 포인터에 대해 유효한 256 워드 u32 영역을 넘겨야 하고, 전역 트위들 테이블은 선언된 256원소 배치를 가져야 한다. 이 전제 아래에서 검사기는 각 명령에 대한 지역 의무를 만들고 이를 해소한다.

-checksafety에 대한 매뉴얼 설명은 출력 해석에 직접 도움이 된다. 이 플래그는 어셈블리를 내보내는 대신 공개 함수의 안전성을 검사하며, 결과는 초기 상태에 대한 안전성 전제에 조건부이다. 매뉴얼이 설명하는 안전성에는 종료, 배열 경계 준수, 유효하고 정렬된 할당 메모리 접근, 산술 연산 인자의 유효성이 포함된다. 따라서 poly_ntt_jazz에 대한 No Safety Violation은 다음과 같이 읽어야 한다. ABI 호출자가 rp에 대해 유효하고 정렬된 다항식 영역을 제공한다면, 그 공개 함수에서 도달하는 모든 호출은 정의되지 않은 Jasmin 동작 없이 종료한다. poly_invntt_jazz도 같은 방식으로 해석되며, poly_basemul_jazz는 여기에 ap와 bp의 유효하고 정렬된 입력 영역을 추가로 요구한다.

2026년 Jasmin 자료는 이 명령을 제약, 소스 assertion, 자동 해소, 그리고 남는 assertion에 대한 EasyCrypt 증명을 결합하는 더 새로운 모듈식 안전성 작업 흐름과 구분한다 [14]. 이 논문이 jasminc -checksafety를 사용하는 이유는 이 스칼라 NTT가 그 정적 검사기에 잘 맞기 때문이다. 기호적 힙 구조, 소스 수준 스택 배열, 데이터 의존 반복 경계가 없고, 단계 특수화 보조 함수들이 검사기가 해소할 수 있는 작은 정수 사실을 그대로 드러낸다. 따라서 성공한 실행은 이 구체적 산출물에 대한 증거이지, 넓은 검사기가 모든 최신 Jasmin 프로그램의 바람직한 방식이라는 주장이 아니다.

가장 눈에 띄는 의무는 배열 경계와 메모리 유효성이다. rp[idx] 접근에 대해 검사기는 rp가 유효한 타입화 영역을 가리킨다는 사실과 현재 추상 상태가 $0 \leq \text{idx} < 256$ 을 함의한다는 사실을 알아야 한다. rp[offset] = coeff 같은 저장에 대해서도 offset의 같은 경계를 증명해야 하며, 포인터가 쓰기 가능한지도 확인해야 한다. 또한 검사기는 워드 수준 연산의 정의성도 추적한다. 캐스트, 부호 있는 해석과 부호 없는 해석, 시프트, 곱셈, 보조 함수 호출은 모두 Jasmin 의미론이 허용하는 방식으로 사용되어야 한다. 따라서 성공

한 안전성 검사는 조건부 명제로 읽어야 한다. ABI 호출자가 유효한 타입화 영역을 제공한다면, 검사된 루틴의 모든 소스 실행은 정의되어 있고 모든 메모리 접근은 선언된 객체 안에 머문다.

이 원리는 스칼라 NTT에 잘 맞는다. 본 코드의 메모리 구조가 단순하기 때문이다. 검사기는 alias 없는 힙 할당, 기호적 바이트 오프셋, 데이터 의존 반복 종료를 다룰 필요가 없다. 단계 특수화 루프와 테이블 인덱스 식을 따라 공개 카운터 범위를 전파하면 된다. 따라서 생성되는 증명 의무는 작은 정수 사실들이며, 각 보조 함수의 리터럴 루프 경계에서 그 출처를 확인할 수 있다.

3.4 상수 시간 검증의 원리

일반 상수 시간 검사는 프로그램의 누출 흔적에 대한 정보 흐름 분석이다. 프로그래머는 공개 함수마다 보안 시그니처를 제공한다. 검사는 그 라벨을 식과 명령을 따라 전파한다. 비밀로 표시된 값을 계산하고, 조합하고, 저장하는 것 자체는 오류가 아니다. 오류는 비밀 의존성이 누출 모델이 관찰 가능하다고 보는 곳에 도달할 때 발생한다. 여기에는 분기 조건, 루프 가드, 메모리 주소, 배열 인덱스, 그리고 선택된 아키텍처 모델이 상수 시간으로 분류하지 않는 명령 피연산자가 포함된다.

Jasmin 문서와 최근 발표 자료는 상수 시간 검증 방법을 둘로 구분한다. 하나는 `jasmin-ct`가 사용하는 비교적 자동화된 타입 시스템 검사이고, 다른 하나는 더 넓은 확률적 상수 시간 성질을 다루는 증명 지향 방법이다. 본 산출물은 전자를 사용한다. 문서화된 타입 규칙은 스칼라 NTT에 필요한 규칙과 정확히 맞는다. 상수는 공개이고, 산술식의 라벨은 피연산자 라벨의 최댓값이다. 배열 인덱스와 원시 메모리 주소는 공개여야 하며, `if`와 `while`의 조건도 공개여야 한다. 공개 함수는 개발용 추론 모드를 쓰지 않는 한 명시적인 `ct` 시그니처를 가져야 한다. 본 코드는 `#declassify` 탈출구를 사용하지 않으므로, 비밀에서 유래한 값이 공개가 되었다는 별도 미검증 주장에 의존하지 않는다 [8, 14].

따라서 “비밀로 분기하거나 비밀로 인덱싱하지 말라”는 구현 규칙은 정확한 검사 의미를 갖는다. 이 NTT 코드에서 다항식 계수는 비밀이고, 트위들 값은 공개 테이블 상수이다. 계수는 `__fqmul`, 덧셈, 뺄셈, 캐스트, 저장을 통해 흐를 수 있다. 그러나 `j < 128, 1 + block, idx, offset`으로는 흐를 수 없다. 그렇게 되면 계수 값이 다른 두 실행이 서로 다른 제어 경로나 주소열을 가질 수 있으므로 검사는 프로그램을 거부한다. 반대로 공개 루프 카운터와 고정 단계 상수는 분기와 주소를 결정해도 된다.

추측 실행 검사는 이 원리를 공개 경계에서 더 강화한다. 문서와 2024-2026년 발표 자료는 이를 Spectre-v1형 일시 실행을 막기 위한 선택적 추측 실행 로드 강화로 설명한다 [6, 14]. 핵심은 암호 코드가 이미 어떤 값이 공개이고 어떤 값이 비밀인지 알고 있으므로, 강화 규율을 일시 실행 동작이 통제되어야 하는 공개 로드와 포인터 값에 집중할 수 있다는 점이다. SCT 타입 시스템은 포인터 값의 보안과 그 포인터가 가리키는 메모리 값의 보안을 분리하고, 정상 실행과 일시 실행도 구분한다. `transient` 포인터는 아키텍처적으로는 공개이지만, 오예측 플래그(MSF) 상태가 초기화되기 전에는 추측 실행 주소 계산에서 자유롭게 공개로 취급되지 않는다. 매뉴얼은 공개 함수 인자가 최소한 `transient`로 취급됨도 설명한다. `export` 함수는 MSF 인자를 받을 수 없으므로,

Jasmin 구성 요소	이 논문에서의 역할
<code>compiler/safetylib</code>	<code>jasminc -checksafety</code> 가 쓰는 추상 해석 기반 안전성 검사 인프라. <code>ct/ sct</code> 시그니처 파서. 이름 붙은 CT 수준, 화살표, 곱, <code>transient</code> , 포인터 및 값 SCT 타입을 포함한다.
<code>compiler/src/securityAnnotations.ml</code>	일반 상수 시간 검사기. 추론 모드를 사용하지 않는 공개 함수는 완전한 주석이 필요하며, 결과 라벨은 입력 라벨로 정당화되어야 한다.
<code>compiler/src/ct_checker_forward.ml</code>	추측 실행 검사기. 직접 및 간접 보안 타입을 모델링하고, 공개 경계 제약과 <code>#init_msf()</code> 가 사용하는 MSF 규율을 다룬다.
<code>compiler/src/sct_checker_forward.ml</code>	<code>jasmin-ct</code> 와 <code>jasmin-ct -speculative</code> 의 명령행 진입점.

표 4: 산출물 해석에 사용한 Jasmin 검사기 구성 요소.

컴파일러는 진입 전에 이미 오예측 상태일 수 있다고 방어적으로 가정하기 때문이다. 그래서 목록 1의 래퍼들은 산술 커널에 들어가기 전에 `#init_msf()`를 호출한다. 문서화된 x86 하향 변환에서 이 연산은 `lfence`를 내보내고 MSF를 Exact 상태로 초기화한다. 그 경계 뒤에는 검사가 포인터 기준 주소를 공개로 타입화할 수 있지만, 포인터가 가리키는 다항식 계수는 계속 비밀로 타입화한다. 따라서 여기서 얻는 주장은 모델 상대적이다. 이 소스와 호출 그래프에 대해 Jasmin SCT 검사기의 Spectre-v1형 누출 모델 안에서 얻은 증거이지, 모든 추측 실행 현상이나 미세아키텍처 채널에 대한 측정 또는 포괄적 증명이 아니다.

본 논의는 특정 작업 환경의 절대 경로가 아니라 제출 산출물 안에서 식별 가능한 구성 요소에 근거한다. 구현은 `jasmin/` 아래에 있고, 원시 재현 로그는 `paper/logs/` 아래에 있으며, 도구 해석에 사용한 Jasmin 문서는 소스 트리 상대 경로로 식별한다. `source/tools/safety_checker.md`, `source/tools/ct.md`, `source/tools/sct.md`, `source/language/semantics/arrays.md`, `source/language/semantics/scalar_types.md`, `source/compiler/passes/index.md`가 특히 중요하다. 아래 명령들은 저장소 리비전 `f0e1399`, Jasmin 컴파일러 버전 2026.03.0으로 재현되었다. 표 4의 검사기 구성 요소도 Jasmin 소스 체크아웃이나 산출물 컨테이너를 기준으로 한 상대 경로로 적었다. 이 점은 중요하다. 이 논문은 Jasmin 검증에 관한 관용적 설명을 독자에게 믿으라고 요구하지 않는다. 매뉴얼은 의도된 도구 인터페이스를 설명하고, 컴파일러 소스는 보안 주석 언어, 공개 함수 주석 검사, 추측 실행 포인터 및 값 타입 규율을 담고 있다.

주석 파서는 목록 1의 표기법을 설명한다. 일반 상수 시간 계약에서 입력 타입은 공개, 비밀, 미지정, 또는 `rp`, `ap`, `bp` 같은 이름 붙은 수준이 될 수 있다. 출력 타입은 그러한 입력 수준들의 집합을 언급할 수 있다. 따라서 `rp * ap * bp -> {ap, bp, rp}`는 다항식에 대한 수학적 곱을 뜻하지 않는다. 반환값이 세 입력 인자에 부여된 보안 수준에 의존할 수 있음을 말하는 보안 시그니처이다. 추측 실행 계약의 경우 `securityAnnotations.ml`은 $\{ptr : \ell_p, val : \ell_v\}$ 꼴의 간접 타입을 정의한다. 또한 `transient`를 정상 실행 성분은 공개, 추측 실행 성분은 비밀인 수준으로 정의한다. 공개 래퍼가 사용하는 진입 타입이 바로 이 타입이다.

현재 구현은 Jasmin 메모리 모델 중 작고 규칙적인 부분만 사용한다. 소스에는 `[p + e]` 형태의 원시 메모리 식도, 소스 수준 스택 배열도 없다. 관찰 가능한 소스 메모리 연산은

타입이 붙은 배열 및 포인터 연산이다.

`rp[idx], rp[offset], ap[i], bp[i],`

여기에 고정 전역 테이블 `jzetas`와 `jzetas_inv` 읽기가 더해진다. 따라서 일반 Jasmin 모델의 바이트 블록 메모리 의무는 워드 배열 의무로 특수화된다. 호출자가 제공하는 다항식 포인터는 유효한 256워드 `u32` 영역을 가리켜야 하며, 계산된 모든 인덱스는 $[0, 256)$ 안에 있어야 한다.

상수 시간 검사는 Jasmin x86 누출 모델을 기준으로 한다. 분기와 메모리 주소는 관찰 가능하다. 모델이 상수 시간으로 분류한 산술 명령은 비밀 의존 누출 의무를 추가하지 않는다. 이 구분은 본 코드에서 특히 중요하다. Montgomery 곱셈은 계수에서 유래한 레지스터 위의 `imul`, `sar` 같은 산술로 낮아진다. 검사기는 선택한 x86 모델 안에서 이 산술을 허용하지만, 같은 계수 유래 값이 분기 조건, 반복 경계, 메모리 인덱스, 또는 아키텍처 모델이 상수 시간으로 분류하지 않는 연산으로 흐르면 거부한다.

4 검사 결과가 설득력을 갖는 이유

검증 논증은 비공식 코딩 관례의 모음이 아니다. 실제 Jasmin 소스에 대해 수행한 세 가지 명시적 검사의 합성이다. 각 검사는 하나의 의미론적 의무에 대응하고, 각 의무는 구현 정리가 잘못 읽히는 서로 다른 가능성을 차단한다.

의도한 정리는 조건부 비간섭 명제로 말할 수 있다. 하나의 공개 루틴을 두 번 호출한다고 하자. 두 호출은 같은 공개 ABI 입력과 같은 타입화된 포인터 유효성 전제를 만족하지만, 포인터가 가리키는 다항식 계수는 서로 다를 수 있다. 두 호출을 Jasmin 의미론으로 실행하고 Jasmin 컴파일러로 컴파일하면, 두 실행은 같은 분기 일정과 같은 메모리 주소 열을 따른다. 쓰이는 계수 값은 비밀 출력이므로 달라질 수 있으나, 관찰 가능한 제어 및 주소 흔적은 계수 값과 무관하다. 안전성 검사는 이 명제의 정의된 실행 전제를 제공한다. 일반 상수 시간 검사는 흔적 비간섭 전제를, 추측 실행 상수 시간 검사는 일시 실행에 대한 공개 진입 경계 전제를 제공한다. 마지막으로 컴파일러 논증이 이러한 소스 성질을 생성된 x86-64 코드와 연결한다.

이 합성의 각 부분은 분리해서 읽어야 한다. 안전성 명제는 `jasminc -checksafety`에서 오며, 유효한 초기 메모리에 조건부이다. 분기와 주소에 관한 시간 흔적 명제는 `jasmin-ct`와 현재 Jasmin 자료가 설명하는 컴파일러의 상수 시간 보존 규율에 근거한다 [14]. 추측 실행 명제는 별도의 SCT 검사기와 그 Spectre-v1형 모델에서 온다. 최근의 Jasmin 컴파일러 front-end 암호학적 보안 보존 연구는 관련성이 크지만, 다른 정리를 증명한다. 그 정리는 KEM의 IND-CCA 보존을 대상으로 하며, 본 산출물에 대해 전체 HAETAE 서명, back-end, 또는 부채널 정리를 곧바로 제공하지 않는다 [4].

이 사슬이 설득력을 가지려면 검사기의 의무가 실제 프로그램 구조와 맞아야 한다. 본 산출물에서는 그렇다. 표 5은 검증 의무와 이를 만족시키는 구체적인 소스 사실을 함께 보인다. 이 의무들은 NTT의 숨은 대수적 성질이나 계수 값에 대한 확률적 가정에 기대지 않는다. 루프, 인덱스, 포인터 타입, 산술 연산의 위치에 관한 구조적 사실이다.

안전성 검사는 정의된 실행을 보장한다. `jasminc -checksafety` 명령은 `compiler/safetylib` 아래의 안전성

검증 의무	산출물에서 성립하는 구체적 이유
다항식 접근은 경계 안에 있다	모든 공개 다항식 포인터의 타입이 <code>ptr u32[256]</code> 이고, 모든 인덱스는 리터럴 단계 크기로 경계가 정해진 공개 카운터이다. 순방향 단계는 테이블 원소 1부터 255까지를 사용하고, 역방향 단계는 0부터 254까지를 사용하며, 역변환 최종 스케일링은 원소 255를 사용한다.
트위들 접근은 경계 안에 있다	모든 루프 가드는 공개 카운터를 128, 64, 또는 <code>HAETAE_N</code> 같은 상수와 비교한다. 주소식에는 포인터 기준 주소, 공개 카운터, 리터럴 단계 오프셋만 등장하며 계수에서 유래한 워드는 등장하지 않는다.
분기는 공개 값에 의해 결정된다 주소는 공개 값에 의해 결정된다	계수와 Montgomery 곱은 레지스터 산술과 저장으로 흐르며, 이것이 의도한 비밀 데이터 경로이다.
비밀 값은 필요한 곳에서 허용된다	각 공개 래퍼는 커널 진입 전에 <code>#init_msif()</code> 를 호출하고, <code>sc</code> 시그니처는 포인터 보안과 포인터가 가리키는 값의 보안을 분리한다.
공개 추측 실행 경계가 명시적이다	

표 5: 검사기 의무와 소스 수준 구현 사실의 대응.

분석을 호출한다. 특히 `safetyInterpreter.ml`은 배열 접근, 부분 배열 접근, 메모리 유효성, 워드 연산 부조건, 반환 조건에 대한 안전성 조건을 만들고, 추상 상태가 그 조건들을 함의할 때에만 `No Safety Violation`을 보고한다. 이 점은 중요하다. 상수 시간 성질은 보통 실행 흔적의 성질인데, 프로그램이 경계 밖으로 나가거나 정의되지 않은 원시 연산을 실행할 수 있다면 흔적 논증은 잘못된 의미론을 대상으로 하게 된다. 그러므로 안전성은 첫 번째 층위이다. 시간 검사가 다루는 모든 흔적이 타입화된 포인터 전체 아래 정의된 저수준 실행에 대응함을 말해 주기 때문이다.

본 산출물에서 생성되는 안전성 의무는 코드와 대조해 수작업으로도 점검할 수 있을 만큼 단순하다. 호출자가 제공하는 유일한 메모리 영역은 다항식 포인터 `rp`, `ap`, `bp`이고, 유일한 전역 배열은 256원소 트위들 테이블이다. 단계 특수화 코드는 모든 배열 인덱스를 경계가 있는 공개 카운터의 아핀 식으로 만든다. 따라서 안전성 검사기가 비밀 데이터에 관한 암호학적 불변식을 발견해야 하는 상황은 없다. 첫 순방향 단계의 $j < 128 \Rightarrow j + 128 < 256$, 기본 곱셈의 `ap[i]`, `bp[i]`, `rp[i]`에 대한 $i < 256$ 같은 산술 사실을 확인하면 된다.

일반 상수 시간 검사는 누출 흔적으로 가는 정보 흐름을 제한한다. `ct_checker_forward.ml`의 일반 검사는 시간 흔적에 영향을 줄 수 있는 프로그램 부분을 위한 타입 시스템이다. 배열 인덱스, 메모리 주소, `if` 조건, `while` 가드, 경계로 쓰이는 식에 대해서는 표현식 검사기가 공개 결과를 요구하는 모드로 호출된다. 아키텍처 모델이 상수 시간이 아니라고 분류한 연산도 마찬가지로 공개 피연산자를 요구한다. 공개 함수에는 추론을 명시적으로 요청하지 않는 한 완전한 보안 주석이 필요하며, 결과 라벨은 입력 시그니처에 등장한 수준만 언급할 수 있다.

따라서 `jasmin-ct` 성공은 단순히 검토자가 비밀 분기를 찾지 못했다는 뜻이 아니다. 검사기가 프로그램을 순회하며 누출 흔적에 기여하는 모든 식이 주어진 보안 환경에서 공개임을 증명했다는 뜻이다. HAETAE NTT 코드에서 비밀 값은 읽어 온 다항식 계수와 그로부터 파생된 산술 값이다. 이 값들은 `+`, `-`, 캐스트, Montgomery 곱셈, 저장으로 흐른다. 반복 가드, 분기 조건, 주소식으로는 흐르지 않는다. 검사기는 바로 이 분리를 강제한다.

추측 실행 상수 시간 검사는 공개 진입 경계를 확인한다. `sct_checker_forward.ml`의 추측 실행 검사기는 정상 실행과 추측 실행의 보안 수준을 구분하고, 포인터 보안과 포인터가 가리키는 값의 보안을 나누어 일반 성질을 강화한다. 로컬 주석 파서는 `transient`를 정상 실행에서는 공개이지만 추측 실행 성분에서는 비밀인 수준으로 정의한다. 따라서

```
{ptr : transient, val : secret}
```

타입의 래퍼 인자는 포인터가 아키텍처적으로는 공개이나, 미세아키텍처 추측 실행 상태가 초기화되기 전의 일시 실행에서는 주의해서 다루어야 함을 뜻한다. 검사기는 MSF 상태도 추적한다. `#init_msf()`는 SCT 검사기에서 특별한 연산이며, 여기서 사용하는 x86 하향 변환에서는 공개 진입 점의 `lfence`에 대응한다.

그러므로 `jasmin-ct -speculative`의 성공은 특정한 잘못된 증명 양식을 배제한다. ABI가 제공한 포인터가 진입부 강화 이전부터 추측 실행 주소 계산에 안전하게 공개라고 가정할 수 없게 한다. `#init_msf()` 이후에는 `_poly_ntt`, `_poly_invntt`, `_poly_basemul`의 본문이 공개 포인터 기준 주소와 공개 카운터만으로 주소를 만들고, 배열에서 읽은 값은 계속 비밀로 취급한다. 이는 공개 암호 루틴에 필요한 정책과 정확히 맞는다.

컴파일러 논증은 소스 검사와 어셈블리를 연결한다. 마지막 단계는 컴파일러를 통한 보존이다. Jasmin 컴파일러는 예측 가능한 저수준 변환을 위해 설계되었고, Jasmin 논문이 설명하는 컴파일러 정확성 개발과 함께 제공된다 [1, 2, 14]. 이 논문의 안전성 및 시간 논증은 그 규율을 사용한다. 소스 수준의 고정 일정은 분기와 인덱스 메모리 피연산자로 낮아질 수 있지만, 컴파일러가 비밀 값으로 인덱싱하는 테이블 조화나 비밀 의존 범위에 대한 분기를 새로 도입해서는 안 된다. 이 설계 선택 때문에 소스 수준 검사가 생성된 `hpoly.s`에 대한 의미 있는 증거가 된다. 주장은 임의의 최적화 컴파일러가 상수 시간성을 보존한다는 것이 아니다. 이 소스가 Jasmin 도구 체인으로 검사되었고, Jasmin 컴파일러의 보존 모델 아래에서 컴파일되었다는 것이다.

따라서 이 논문이 사용하는 합성 명제는 다음과 같은 산출물 정리 도식으로 읽어야 한다. 첫째, `jasminc-checksafety`는 타입화된 포인터 전제 아래 세 공개 소스 루틴의 실행이 정의됨을 증명한다. 둘째, `jasmin-ct`는 분기와 메모리 주소로 이루어진 누출 흔적이 계수 값과 독립임을 증명한다. 셋째, `jasmin-ct -speculative`는 같은 소스를 Jasmin SCT 검사기의 `transient` 포인터 모델에 대해 검사한다. 넷째, `jasminc`는 그 소스를 스칼라 x86-64 어셈블리로 컴파일하며, 재생성한 어셈블리는 제출된 `hpoly.s`와 바이트 단위로 같다. 결론은 정확히 이 전제들에 조건부이다. 제출 어셈블리는 검사된 소스의 컴파일러 출력이고, 그 안전성 및 시간 주장은 Jasmin 모델이 보존하는 주장이다. 별도의 바이너리 분석 정리가 아니며, 손으로 수정한 어셈블리로 확장되지 않는다.

이 구현에 대해 왜 설득력 있는가. 구현은 검사 의무와 암호 구현자가 기대하는 비공식 규율이 일치하도록 작성되어 있다. 데이터 의존 루프, 데이터 의존 조건문, 원시 소스 수준 포인터 산술, 계수로 인덱싱되는 테이블 조화가 없다. 주소에 영향을 주는 변수는 공개 카운터와 리터럴 단계 상수뿐이다.

비밀을 담은 변수는 계수와 그 계수에서 파생된 산술 임시 값뿐이다. 따라서 검사기 출력은 프로그램을 모른 채 붙인 인증서가 아니다. 공개 일정과 공개 주소, 그리고 비밀 산술 값이라는 단순한 불변식을 확인하는 결과이다.

5 안전성 논증

안전성 증명은 모든 메모리 위치가 리터럴 경계를 가진 공개 카운터로 선택된다는 사실에 근거한다. 단계 길이가 ℓ 인 순방향 단계의 보조 함수는 다음과 같은 도식적 형태를 가진다.

$$\text{block} < \frac{256}{2\ell}, \quad j < \ell, \quad \text{start} = \text{block} \cdot 2\ell,$$

그리고

$$\text{idx} = \text{start} + j, \quad \text{offset} = \text{idx} + \ell.$$

따라서 검사기는 지역적인 산술만으로

$$0 \leq \text{idx} < 256 \quad \text{and} \quad 0 \leq \text{offset} < 256$$

을 증명하면 된다. 첫 순방향 보조 함수가 가장 큰 경우이다. $\text{block} < 1$, $j < 128$, $\text{offset} = j + 128$ 이다. 나머지 보조 함수는 더 작은 단계 길이와 더 많은 블록에 대해 같은 형태를 인스턴스화한다. 역방향 보조 함수도 반대 단계 순서로 같은 주소 구조를 사용한다. 역변환 최종 스케일링 루프는 단 하나의 의무, 즉 $0 \leq j < 256$ 만 가진다.

트위들 테이블 접근도 구성상 고정되어 있다. 순방향 변환은 `jzetas[1]`부터 `jzetas[255]`까지를 사용한다. 역방향 버터플라이 단계는 `jzetas_inv[0]`부터 `jzetas_inv[254]`까지를 사용하며, 역변환 최종 스케일링은 `jzetas_inv[255]`를 읽는다. 모두 256원소 테이블 타입 안에 있다.

기본 곱셈은 더 단순하다. 공개 루프 카운터 i 는 0부터 255까지 움직이고, 소스 수준 메모리 연산은 `ap[i]`, `bp[i]`, `rp[i]`뿐이다. 래퍼 타입이 세 포인터 모두에 `ptr_u32[HAETAE_N]`을 부여하고 `HAETAE_N = 256`이므로, 안전성 증명은 루프 경계 불변식 $0 \leq i < 256$ 으로 환원된다. `ap`와 `bp`의 `const` 한정자도 의도한 데이터 흐름과 맞다. 입력 다항식은 읽기만 하고, `rp`만 쓴다.

6 상수 시간 논증

일반 상수 시간 검사는 CCS식 누출 규율을 구문적으로 강제한다. 분기, 루프 가드, 메모리 주소, 배열 인덱스를 결정하는 모든 식은 공개여야 한다. 계수와 Montgomery 곱은 비밀일 수 있지만 흔적을 제어해서는 안 된다.

순방향 및 역방향 NTT에서 계수 유래 값은 모두 산술 위치에 머문다. 읽어 온 계수는 더해지고, 빼지고, 캐스트되고, 시프트되고, Montgomery 산술에서 곱해질 수 있으며, 결과는 다항식 셀에 다시 저장될 수 있다. 그러나 그 값은 분기 조건, 루프 경계, 주소식으로 흐르지 않는다. 루프 일정은 단계 상수로 고정되어 있고, 트위들 테이블 인덱스는 $1+\text{block}$, $64+\text{block}$, $254+\text{block}$ 같은 공개 식이다.

기본 곱셈의 흔적은 목록 4에 보인 공개 256회 `load/load-store` 패턴이다. 읽어 온 `a`와 `b`는 비밀이고, 곱 `r`도 비밀이다. 그러나 분기 조건은 $i < 256$ 이며, 모든 메모리 주소는 같은

공개 인덱스 i 를 사용한다. 따라서 계수 값은 rp 에 기록되는 워드만 결정하고, 어떤 명령이 실행되는지 또는 어느 위치가 접근되는지는 결정하지 않는다.

공개 래퍼의 일반 ct 주석은 다항식 데이터를 공개로 만들지 않으면서 결과의 보안 의존성을 기록한다. `poly_ntt_jazz`와 `poly_invntt_jazz`의 $rp \rightarrow rp$ 주석은 반환 포인터 및 결과가 입력 다항식 인자와 같은 보안 의존성을 가진다고 말한다. `poly_basemul_jazz`에서는

```
rp * ap * bp -> {ap, bp, rp}
```

주석이 rp, ap, bp 를 이름 붙은 보안 수준으로 사용한다. 결과 라벨은 보수적이다. 소스 본문은 목적지 포인터를 반환하지만, 반환값이 세 입력 라벨 모두에 의존할 수 있다고 허용한다. 이 라벨의 최소성은 핵심 상수 시간 의무와 별개의 문제이다. 검사기는 여전히 `ap[i]`, `bp[i]`, `rp[i]`의 인덱스가 공개임을 요구한다.

7 추측 실행 상수 시간

추측 실행 검사기는 일시 실행에 대해 일반 성질을 강화한다. 본 구현은 포인터의 보안 수준과 포인터가 가리키는 값의 보안 수준을 구분하는 `sct` 주석을 사용한다.

```
{ptr :  $\ell_p$ , val :  $\ell_v$ }.
```

공개 래퍼는 비밀 다항식 값에 대한 `transient` 포인터를 입력으로 받고, 비밀 다항식 값에 대한 공개 포인터를 반환한다. 직관적으로, 진입부의 추측 실행 경계 이후에는 포인터 기준 주소를 공개 주소로 쓸 수 있지만, 계수 값은 끝까지 비밀로 남는다.

`#init_msf()`가 강화 경계이다. 검사기 안에서는 미세아키텍처 추측 실행 상태를 초기화하고, 여기서 사용하는 x86-64 하향 변환에서는 공개 어셈블리 진입부의 `lfence`로 나타난다. 그 뒤 NTT와 기본 곱셈 본문은 공개 포인터 기준 주소와 공개 카운터만으로 주소를 만든다. 다항식 셀에서 읽은 비밀 값은 산술과 저장으로 흐르며, 일시 실행 주소 계산에는 쓰이지 않는다.

이 성질은 공개 ABI 경계에서 의미가 있다. 일반 상수 시간 증명은 커밋된 분기와 커밋된 주소로부터의 누출을 배제한다. 추측 실행 증명은 공개처럼 보이는 포인터나 경계가 진입 fence 이전에도 공개라고 조용히 가정하는 일을 막는다. 이 구현의 다항식 커널 안에는 별도로 강화가 필요한 비밀 의존 경계가 없다. 이 호출 그래프에 대해 Jasmin SCT 모델이 요구하는 추측 실행 경계는 공개 진입점에 있다. 이는 그 모델 밖의 모든 추측 실행 변종, 데이터 의존 프리패치, 주파수 효과, 기타 미세아키텍처 채널을 포괄한다는 뜻이 아니다.

8 컴파일러 수준 보증

위 검사는 손으로 작성한 어셈블리 증명이 아니라 `jasmin/hpoly.jazz`에 대해 수행된다. 이것이 의미를 갖는 이유는 Jasmin이 소스 수준 규율을 보존하는 예측 가능한 컴파일러를 중심으로 설계되었기 때문이다 [1, 2, 14]. Jasmin 컴파일러는 상수 시간 코드에 위험한 최적화를 피한다. 예를 들어 비밀 산술을 비밀 인덱스 테이블 조회로 바꾸거나, 비밀 의존 값에 대한 `case split`을 새로 도입하지 않는다. 공개 루프

명령	결과
<code>jasminc -checksafety -o /tmp/hpoly-safe.s jasmin/hpoly.jazz</code>	<code>poly_ntt_jazz</code> , <code>poly_invntt_jazz</code> , <code>poly_basemul_jazz</code> 에 대해 safety violation 없음.
<code>jasmin-ct jasmin/hpoly.jazz</code>	일반 상수 시간 검사 통과.
<code>jasmin-ct -speculative jasmin/hpoly.jazz</code>	추측 실행 상수 시간 검사 통과.
	공개 래퍼는 <code>transient</code> 포인터 입력과 공개 포인터, 비밀 값 출력을 가진다.

표 6: 컴파일러 수준 검증 증거.

구조는 분기와 인덱스 메모리 피연산자로 낮아질 수 있지만, 비밀 값이 분기 predicate나 주소 선택자로 바뀌어서는 안 된다. 따라서 형식적 주장은 소스 수준 검사 결과와 Jasmin 컴파일러의 보존 정리 및 시간 규율의 합성이다. 아래의 어셈블리 점검은 생성 코드가 기대한 공개 카운터 구조를 갖는지 확인하는 기본 점검이다.

이 논문은 최신 front-end 암호학적 보안 보존 정리를 HAETAE에 대한 전제로 사용하지 않는다. 그 정리는 소스 수준 암호 증명과 컴파일된 코드 사이의 간극을 다루는 관련 연구로 인용한다. 반면 이 논문의 결과는 스칼라 NTT 커널에 대한 소스-어셈블리 안전성 및 시간 흔적 논증이다.

생성된 스칼라 어셈블리는 기대한 형태를 가진다. 각 공개 래퍼에는 진입부 `lfence`가 있다. 기본 곱셈 본문은 `ap[i]`, `bp[i]`, `rp[i]`를 공개 루프 카운터를 `scaled index`로 쓰는 `base-plus-index` 메모리 피연산자로 낮추며, 루프 분기는 그 카운터를 상수 256과 비교한다. NTT 단계 루프도 공개 카운터로 분기하고, 공개 기준 포인터, 공개 단계 상수, 공개 카운터로 주소를 만든다. 계수는 산술 레지스터와 저장되는 워드에 나타나지만, 분기 predicate나 주소 레지스터에는 나타나지 않는다.

따라서 컴파일러 수준 보증은 조건부 합성으로 읽어야 한다. 소스 수준 안전성, 소스 수준 상수 시간 검사, 추측 실행 진입부 강화, Jasmin의 보존 규율이 함께 생성된 어셈블리를 검증된 스칼라 구현으로 사용할 근거를 제공한다. 이 주장은 수작업 어셈블리 스타일 감사보다 강하다. 소스 계약과 컴파일러 불변식에 묶여 있기 때문이다. 동시에 이 주장은 산출물이 식별한 Jasmin 의미론, 검사기 모델, 컴파일러 버전에 상대적인 주장으로 남는다.

9 검증 증거

표 6의 명령을 산출물 위에서 실행했다. 안전성 검사기는 세 공개 루틴 모두에 대해 safety violation이 없다고 보고한다. 일반 상수 시간 검사는 성공적으로 종료한다. 추측 실행 검사기도 성공적으로 종료하며, 내부 커널과 공개 래퍼에 대해 기대한 공개 포인터 및 비밀 값 시그니처를 출력한다.

재현성을 위해, 보고한 실행은 `jasminc -version = Jasmin Compiler 2026.03.0`과 저장소 리비전 `f0e1399`를 사용했다. 원시 로그는 `paper/logs/`에 포함되어 있다. `jasminc-version.log`, `git-revision.log`, `jasmin-checksafety.log`, `jasmin-ct.log`, `jasmin-ct-speculative.log`, `jasmin-compile.log`가 해당한다. `jasminc jasmin/hpoly.jazz -o /tmp/hpoly-after.s`로 생성한 어셈블리는 검사 대상인 `hpoly.s`와 바이트 단위로 같았다. 산출물은 이를 `assembly-sha256.log`와 `bin assembly-diff.log`로 기록

산출물 증거	확인되는 내용
jasmin-checksafety.log	세 공개 함수가 모두 No Safety Violation 을 보고한다. 기본 곱셈 출력은 mem_rp, mem_ap, mem_bp를 명명한다.
jasmin-ct-speculative.log	내부 커널은 공개 포인터 기준 주소와 비밀 값을 갖는 타입으로 검사된다. 공개 래퍼는 transient 포인터를 입력으로 받고 비밀 값에 대한 공개 포인터를 반환한다.
hpoly.s	각 공개 래퍼는 lfence로 시작한다. 루프는 공개 카운터를 상수와 비교하고, 메모리 피연산자는 base-plus-scaled-index 형태이다.
assembly-sha256.log, assembly-diff.log	재생성한 어셈블리와 검사 대상 hpoly.s의 SHA-256 해시가 같고, 기록된 diff는 비어 있다.

표 7: 검사기 및 어셈블리 주장 점검에 사용한 원시 증거.

검사 표면	크기	검사상 역할
상수와 테이블	소스 69줄	params.jinc와 zetas.jinc는 단계 특수화 루프가 사용하는 q , n , 트위들 배치를 고정한다.
산술 보조 함수	소스 34줄	reduce.jinc는 __montgomery_reduce와 __fqmul을 제공한다. 검사는 그 산술 결과가 분기와 주소로 흘러가지 않음을 확인한다.
다항식 커널	소스 698줄	poly.jinc에는 순방향 여덟 단계, 역방향 여덟 단계, 최종 스케일링, 기본 곱셈이 들어 있다.
ABI 래퍼	소스 38줄	hpoly.jazz는 검사 대상 세 루틴을 공개하고 상수 시간/추측 실행 상수 시간 경계 annotation을 붙인다.
생성 어셈블리	어셈블리 908 줄	hpoly.s는 세 공개 래퍼를 포함하며, 각 래퍼는 진입점 lfence와 컴파일된 단계 루프를 가진다.
원시 검증 로그	로그 211줄	paper/logs/*.log는 컴파일러 버전, 소스 리비전, 검사기 출력, 어셈블리 해시, 빈 어셈블리 diff를 기록한다.

표 8: 검증 산출물의 크기와 검사 표면. 이 표는 감사용 요약이며, 실행 시간 benchmark가 아니다.

한다. 제출 산출물에는 정확한 Jasmin 컴파일러 소스 또는 컨테이너 이미지도 식별되어야 한다. 안전성 및 시간 주장은 이 명령에 사용한 검사기 구현에 상대적이기 때문이다.

표 7는 원시 로그와 생성 어셈블리를 대조해 확인한 증거를 요약한다. 이 표가 로그를 대체하지는 않는다. 다만 어떤 구체적 산출물이 어떤 주장을 뒷받침하는지를 적어, 논증이 산문 설명에만 의존하지 않도록 한다.

저장소 루트에서의 최소 재현 절차는 네 단계로 충분하다. 먼저 `jasminc -version`과 `git rev-parse --short HEAD`로 도구 버전과 소스 리비전을 기록한다. 다음으로 세 소스 검사를 실행한다. `jasminc -checksafety -o /tmp/hpoly-safe.s jasmin/hpoly.jazz, jasmin-ct jasmin/hpoly.jazz, jasmin-ct -speculative jasmin/hpoly.jazz`가 그 명령이다. 셋째, `jasminc jasmin/hpoly.jazz -o /tmp/hpoly-after.s`로 어셈블리를 다시 생성한다. 마지막으로 `hpoly.s`와 새로 생성한 파일을 SHA-256 및 일반 diff로 비교한다. 기대되는 결과는 `paper/logs/`에 기록된 내용과 같다. 안전성 위반이 없고, 일반 및 추측 실행 상수 시간 검사가 성공하며, 어셈블리 해시가 같고 diff가 비어 있어야 한다.

이 표는 어떤 경우에 증거를 다시 생성해야 하는지도 정한다. `hpoly.jazz`, `poly.jinc`, `reduce.jinc`, `zetas.jinc`, `params.jinc`가 바뀌면 검사된 소스 표면이 바뀐다. Jasmin 컴파일러 버전이 바뀌면 검사기 구현이 바뀐다. `hpoly.s`가

바뀌면 `fence` 배치와 메모리 주소 규율을 인용하는 어셈블리 객체가 바뀐다. 이런 변경은 안전성, 일반 상수 시간, 추측 실행 상수 시간, 어셈블리 비교 로그를 다시 만들기 전까지 기존 증거를 무효로 한다. 또한 원시 메모리 연산, 데이터 의존 분기, 비밀 해제(declassification), 또는 상응하는 상수 시간/추측 실행 상수 시간 시그니처가 없는 새 래퍼를 추가한다면 명령을 다시 실행하는 것만으로는 충분하지 않고, 누출 논증 자체를 새로 검토해야 한다.

기본 곱셈에 대한 안전성 출력은 특히 유용하다. 출력은 세 메모리 인자

`mem_rp, mem_ap, mem_bp`

를 명명한다. 성공한 검사는 타입화된 입력 포인터 가정만으로 `ap/bp`의 모든 읽기와 `rp`의 모든 쓰기가 안전함을 증명하기에 충분하다는 사실을 확인한다. NTT 래퍼에 대해서도 출력은 하나의 다항식 포인터 인자 `rp` 아래 안전성을 확인한다.

추측 실행 상수 시간 출력은 공개 래퍼가 `transient` 포인터 입력에서 경계를 설정한 뒤 내부 커널에 다음 폴의 타입을 부여한다.

```
{ptr : public, val : secret}
→
{ptr : public, val : secret}.
```

`poly_basemul_jazz`의 세 입력 인자는 모두

```
{ptr : transient, val : secret}
```

타입이고, 출력은

```
{ptr : public, val : secret}
```

타입이다. 이는 의도한 정책과 맞다. 포인터 기준 주소는 주소 계산을 위해 공개가 되지만, 다항식 내용은 계속 비밀이다.

10 범위와 신뢰 가정

결과의 범위는 의도적으로 제한되어 있다. 이 논문은 산출물의 스칼라 Jasmin 루틴을 다루며, AVX2 구현이나 전체 HAETAE 서명 스킴을 다루지 않는다. 또한 Jasmin 소스 및 컴파일러 모델 안에서 안전성과 상수 시간 성질을 증명한다. 모든 물리적 미세아키텍처 효과에 대한 측정 주장이 아니다. 호출자가 공개 루틴의 타입화된 포인터 전제를 만족한다고 가정한다. 즉, 입력 포인터는 유효한 256워드 다항식 영역을 가리켜야 하며, ABI는 생성 어셈블리를 그 시그니처에 맞게 호출해야 한다.

이 주장은 기능적 정확성과도 분리된다. 동반 EasyCrypt 개발은 현재 추출된 순방향 및 역방향 NTT 래퍼가 추상 변환과 일치함을 증명한다. `poly_basemul_jazz`에 대해서는 아직 완전한 end-to-end 기능적 정확성 증명이 포함되어 있지 않다. 그 루틴에 대해 이 논문이 주장하는 것은 안전성과 시간 규율이다. 안전성과 상수 시간만으로 변환이 수학적으로 의도한 NTT임을 보일 수는 없다. 그것들은 컴파일된 구현이 모든 계수 값에 대해 정의된 실행을 갖고, 분기 및 메모리 주소 흔적이 고정됨을 보인다. 반대로 추출 모델에

대한 기능적 증명도 구현 주장이 되려면 이 컴파일러 수준 증위를 필요로 한다.

기능적 증명의 경계는 조금 더 정확히 밝힐 필요가 있다. 이 논문에서 검사한 Jasmin 소스는 안전성 검사를 위해 보조 함수로 나뉜 형태이기 때문이다. 동반 EasyCrypt 체인은 먼저 이 보조 함수 기반 소스를 Hpoly_extract.ec로 추출한다. CTLoopEquiv.ec의 pRHL 브리지는 이 helperized 추출 결과를 증명용 직접 루프 모델 Hpoly_loop.ec와 연결한다. 이어서 refinement 증명은 직접 루프 모델을 NTT_Fq.ec의 명령형 명세와 연결하고, NTTFullSpec.ec 및 NTTFullAlgebra.ec는 그 명령형 명세를 추상 full NTT 및 inverse NTT와 연결한다. 최상위 파일 NTTEndToEnd.ec는 이 단계를 합성하여 poly_ntt_jazz와 poly_invntt_jazz에 대한 정리를 제공한다. 따라서 기능적 주장은 안전성 및 시간 검사에 쓰인 것과 같은 helperized 소스 형태를 대상으로 하지만, 두 NTT 래퍼에 한정되며 명시된 표현 관계와 입력 bound 전체 아래 성립한다.

이 주장은 최근 interaction tree와 RHL에 기반한 컴파일러의 암호학적 보안 보존 정리와도 구분된다. 그 연구는 Jasmin 컴파일러 front-end가 KEM IND-CCA 보안을 보존한다는 정리를 다룬다 [4]. 이 논문은 그 정리를 인스턴스화하지 않고, 전체 HAETAE 서명 스킴을 다루지 않으며, 생성 어셈블리에 대한 암호학적 보안 reduction을 도출하지 않는다. 여기서 제공하는 것은 그러한 상위 주장을 이 스칼라 코드에 대한 주장으로 읽기 전에 필요한 하위 구현 증위이다.

11 관련 연구

Jasmin은 고속 고신뢰 암호 코드 작성을 위한 언어와 검증 컴파일러 방법론을 제시했다 [1]. Last Mile 연구는 Jasmin과 EasyCrypt를 결합하여 최적화된 저수준 구현을 기능적 정확성 및 부채널 논증과 연결할 수 있음을 보였다 [2]. 이후 Jasmin 발표 자료들은 소스 수준 추론, 자동 안전성 및 상수 시간 검사기, EasyCrypt 지원, 추측 실행 보안에 대한 진행 중인 작업을 포함하는 workbench 관점을 문서화한다 [11, 8, 6, 14]. 이 논문은 Jasmin 이론이나 검사기 구현을 확장하지 않는다. 대신 산출물 수준의 한 인스턴스를 제공한다. 동적인 NTT 일정을 고정 보조 함수로 표현하여 기존 안전성, 일반 상수 시간, 추측 실행 상수 시간 검사기가 구체적인 HAETAE 소스를 처리할 수 있게 한 것이다.

컴퓨터 보조 암호학은 암호 명세와 실행 코드를 연결하는 여러 방법을 발전시켜 왔다 [5]. HACLS*는 F* 기반 도구 체인으로 이식 가능한 암호 구현을 검증한다 [15]. Fiat-Crypto는 고수준 명세로부터 산술 코드를 생성하고 증명한다 [7]. 최근 Kyber 연구는 Jasmin과 증명 도구를 사용해 양자내성 산술 구현의 정확성을 다룬다 [3]. 본 연구의 범위는 더 좁다. 일반적인 코드 생성 방법이나 전체 암호 라이브러리가 아니라, 하나의 스칼라 HAETAE NTT 산출물에 대한 안전성 및 시간 측면을 보고한다. 이 좁은 범위는 의도된 것이다. 기능적 EasyCrypt 증명이 생성된 x86-64 구현에 관한 주장으로 읽히려면 먼저 기대야 하는 컴파일러 수준 증위를 분리해 제시하기 때문이다.

Jasmin의 최신 컴파일러 보안 연구는 다른 간극을 다룬다. 그 연구는 interaction tree로 확률적 암호 보안을 형식화하고, KEM IND-CCA 보안을 중심으로 컴파일러 front-end 보존을 증명한다 [4]. 이 결과는 본 산출물과 상보적이다. 정확한 소스-어셈블리 명제의 필요성을 잘 보여 주지만, 이

HAETAE NTT 산출물이 실제로 제공하는 안전성, 상수 시간, SCT, 기능적 정확성 증거를 따로 구분해 써야 한다는 요구를 없애지는 않는다.

12 결론

스칼라 HAETAE NTT 구현은 보안 관련 구조가 Jasmin 컴파일러 도구 체인에 잘 드러나도록 작성되었다. 단계 특수화는 다항식 및 테이블 경계를 안전성 검사기가 볼 수 있게 한다. 일반 상수 시간 주석은 계수 값이 비밀일 수 있지만 분기나 주소를 구동할 수 없음을 표현한다. 추측 실행 주석과 #init_msf()는 포인터 기준 주소에 대해 공개 진입 경계를 세우면서 다항식 내용은 계속 비밀로 유지한다.

그 결과 산출물은 세 공개 스칼라 루틴에 대해 Jasmin 안전성 검사, 일반 상수 시간 검사, 추측 실행 상수 시간 검사를 모두 통과한다. 여기에 Jasmin 컴파일러의 보존 규율을 결합하면 구체적인 소스에서 어셈블리까지의 보증이 얻어진다. 생성된 스칼라 x86-64 코드는 타입화된 포인터 전체 아래 안전하며, 비밀 다항식 계수와 무관한 고정 분기 및 메모리 주소 일정을 가진다. 이 컴파일러 수준 결과는 현재 순방향 및 역방향 NTT 래퍼에 대해 제공되는 별도의 EasyCrypt 기능적 증명을 보완하지만, 이를 대체하지는 않는다.

따라서 이 논문은 초점을 좁힌 검증 산출물 논문으로 제출하는 것이 가장 정확하다. 구현 대상이 분명하고, 검사 기록이 재현 가능하며, 누출 모델과 신뢰 경계가 명시되어 있기 때문이다. 반면 HAETAE 다항식 산술 전체에 대한 완전한 구현 정확성 논문으로 자리매김하려면 기본 곱셈의 기능적 증명, 전체 통합 정리, 성능 비교가 추가되어야 한다. 이들은 이 논문의 안전성 및 시간 결과가 몰래 기대는 전체가 아니라, 기여를 확장하는 다음 단계이다.

참고문헌

- [1] J. B. Almeida, M. Barbosa, G. Barthe, A. Blot, B. Gregoire, V. Laporte, T. Oliveira, H. Pacheco, B. Schmidt, and P.-Y. Strub. Jasmin: High-assurance and high-speed cryptography. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1807–1823. ACM, 2017.
- [2] J. B. Almeida, M. Barbosa, G. Barthe, B. Gregoire, A. Koutsos, V. Laporte, T. Oliveira, and P.-Y. Strub. The last mile: High-assurance and high-speed cryptographic implementations. In *2020 IEEE Symposium on Security and Privacy*. IEEE, 2020.
- [3] J. B. Almeida, M. Barbosa, G. Barthe, B. Gregoire, V. Laporte, J.-C. Lechenet, T. Oliveira, H. Pacheco, M. Quaresma, P. Schwabe, A. Sere, and P.-Y. Strub. Formally verifying kyber: Episode IV: Implementation correctness. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(3):164–193, 2023.
- [4] S. Arranz-Olmos, G. Barthe, L. Blatter, B. Gregoire, V. Laporte, and P. Torrini. The jasmin compiler preserves cryptographic security: Formalizing cryptographic security with itrees and proving compiler

preservation with rhl. arXiv:2511.11292, 2025. Version 1, 14 November 2025.

- [5] M. Barbosa, G. Barthe, K. Bhargavan, B. Blanchet, C. Cremers, K. Liao, and B. Parno. Sok: Computer-aided cryptography. In *42nd IEEE Symposium on Security and Privacy*, pages 777–795. IEEE, 2021.
- [6] Engineering Cryptographic Software course. Engineering cryptographic software: High-assurance crypto and jasmin. Course slides, 2024.
- [7] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala. Simple high-level code for cryptographic arithmetic: With proofs, without compromises. In *2019 IEEE Symposium on Security and Privacy*, pages 1202–1219. IEEE, 2019.
- [8] B. Gregoire. The jasmin workbench for high-assurance and high-speed cryptography. Presentation slides, June 2023.
- [9] B. Gregoire and S. Priya. Latest improvements in the jasmin compiler and protection against spectre attacks. Presentation slides, June 2022.
- [10] HAETAE Team. HAETAE: Lattice-based digital signature algorithm. Submission to the Korean Post-Quantum Cryptography Competition, 2022. <https://kpmc.or.kr/competition.html>.
- [11] V. Laporte and the Jasmin contributors. Jasmin: a certified workbench for high-assurance and high-speed cryptography. Coq Workshop slides, July 2021.
- [12] P. L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.
- [13] J. M. Pollard. The fast Fourier transform in a finite field. *Mathematics of Computation*, 25(114):365–374, 1971.
- [14] P. Schwabe. Engineering cryptographic software: The jasmin framework. Course slides, Jan. 2026.
- [15] J. K. Zinzindohoue, K. Bhargavan, J. Protzenko, and B. Beurdouche. HACl*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1789–1806. ACM, 2017.